Ejercicios básicos – Repaso para examen

Ejercicio 1

Crea una clase llamada **Password** que siga las siguientes condiciones:

- Que tenga los atributos longitud, contraseña y día/hora de creación. Por defecto, la longitud será de 8.
- Los constructores serán los siguiente:
  - Un constructor por defecto que pida la longitud del Password por teclado. La longitud debe estar entre 8 y 16 caracteres. En caso de no ser válida, deberá volver a pedirse hasta que tenga una longitud válida.
  - Un constructor con la longitud que le pasemos por parámetro. En caso de recibir una longitud no válida avisará y le asignará el valor por defecto (8).
- Ambos constructores generarán una contraseña aleatoria con esa longitud y almacenarán como día/hora de creación la fecha y hora en que se ha creado el objeto. Si quieres, puedes visualizar la contraseña generada en el constructor.
- La clase dispondrá de los siguientes métodos de acceso:
  - Métodos get para consultar la contraseña, la longitud y el día/hora de creación.
- Crea una propiedad para modificar la longitud de la contraseña. En caso de usarla y realizar un cambio de longitud, la contraseña volverá a ser regenerada aleatoriamente y cambiará. Debes controlar también que la contraseña tenga una longitud válida (entre 8 y 16 caracteres), ya que en caso contrario se le asignará el valor por defecto 8.
- Los métodos que implementa serán:
  - **EsFuerte()**: devuelve un booleano si el Password es fuerte o no. Para que sea fuerte debe tener más de 2 mayúsculas, más de 1 minúscula y más de 5 dígitos.
  - **GenerarPassword()**: genera una contraseña aleatoria del objeto con la longitud que tenga. Piensa un algoritmo adecuado para que te pueda generar de forma aleatoria una contraseña que incluya caracteres alfanuméricos (letras, dígitos, signos, etc…). Piensa en ayudarte del código ASCII (caracteres imprimibles entre el 32 y 126).
- Decide, en los dos métodos anteriores si deben ser de interface o de implementación.

Posteriormente, en el **Main()** de la Clase **Program**:

- Crea un array de Passwords con el tamaño que se le indique. Debe pedirse el valor de dicho tamaño previamente por teclado.
- Crea objetos de clase Password y almacénalo en cada posición del array. Utiliza para ello el primer constructor programado en la clase Password (el que pide la longitud por teclado).
- Crea otro array de booleanos donde se almacene si el password del array de Password es o no fuerte.
- Para finalizar, muestra la contraseña de cada password indicando si es o no fuerte y la fecha/hora de creación de la misma. Usa este formato simple:
  - Contraseña: valor --- booleano --- Fecha/horaCreación.
  - Contraseña: valor --- booleano --- Fecha/horaCreación.
  - …..

Utiliza la programación más eficiente para poder lograr e implementar todas las cuestiones indicadas en los puntos anteriores.

Ejercicio 2

Crear una clase llamada **Serie** con las siguientes características:

- Sus atributos serán número de identificación de la serie, título, numero de temporadas, estrenada, fecha de estreno, género y creador.
- El número de identificación de la serie debe ser calculado automáticamente y de forma autoincrementativa cada vez que se instancie un nuevo objeto de la clase Serie. El primer número de identificación será el valor 100 y el resto, según se vayan creando objetos de tipo Serie, será correlativo (101, 102, 103, etc…).
- Por defecto, el número de temporadas es de 3 temporadas y la serie no estará estrenada. El resto de los atributos serán valores por defecto según el tipo del atributo que tenga, salvo el identificador que seguirá la normativa especificada en el punto anterior. Si no está estrenada, la fecha de estreno deberá tener el valor mínimo que admita la clase DateTime.
- Los constructores que se implementarán serán:
  - Un constructor por defecto que pida la información necesaria de la Serie por teclado. En caso de no estar estrenada, no se deberá introducir fecha de estreno. No se deben admitir número de temporadas menor a 1.
  - Un constructor con el título, género y creador. Resto de atributos tendrán valor por defecto.
- Los métodos de acceso que se implementara serán:
  - Métodos get de todos los atributos. De forma específica, el getter del atributo estrenada debe devolver “Estrenada” o “No estrenada”.
  - Propiedades con sólo la parte set para todos los atributos a nivel de objeto, excepto del número de identificador, si está estrenada o no y fecha de estreno.
- Crea los siguientes métodos de interface:
  - VisualizaInformación(): que muestre la información básica de todos los atributos de una Serie.
  - RealizarEstreno(): que, en caso de no estar estrenada la serie, cambia su estado a estrenada y le asigna como fecha de estreno la fecha actual del sistema.

Posteriormente, en el **Main()** de la Clase **Program**:

- Crea una Lista de Series, con 3 series. La primera serie creada con el primer constructor, y las otras dos con el segundo constructor. Para estas dos últimas series elige tú la información a suministrar al constructor de forma literal.
- Visualiza para cada Serie de la lista únicamente su nombre y si ha sido estrenada o no. Usa este formato simple, como ejemplo:
  - Serie: Juego de Tronos --- Estrenada.
  - Serie: Walking dead. El retorno --- No estrenada.
  - …..

- Desarrolla tres funciones en el Main() denominadas:
  - SerieConMasTemporadas que reciba toda la lista de Series y devuelva la serie que tiene más temporadas. En caso de que hayan varias series con el mismo número de temporadas máximas debe devolver la última introducida en la lista.
  - GeneroSerie, que reciba como parámetro la lista de Series y el nombre de una serie y devuelva el género de dicha serie en caso de estar en la lista de Series o el valor NULL en caso de no existir en la lista de Series.
  - DiasEstreno, que reciba el identificador de una serie, junto con la lista de series y devuelva los días que hace que la serie se ha estrenado desde la fecha actual. En caso de no estar estrenada devolverá -1.